Rehydration Process - Horizontal Scaling

Currently prepare and rehydration process is triggered via API endpoint and the request will be processed by a single instance in API app service.The process runs in single instance with multi-threaded , controlled by wait time(throttleMs) for processing and no of documents(takePerThrottle) per batch of data from Cosmos fetch operation

Prepare Stage

- This has to complete before #2 should be kicked off.
- This is a background task and the endpoint will return an immediate `202 Accepted`.

Functionality:

- This method process the items from the states queue asynchronously, making the records for rehydration by updating the property **rehydrateRequired** as true.

End Points:

- POST request DEV
  URL- https://eheu2dv1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/prepare/all
  Content-Type: application/json
  Settings - {"throttleMs":5,"takePerThrottle":5}
- POST request INTG
  URL- https://eheu2in1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/prepare/all
  Content-Type: application/json
  Settings - {"throttleMs":5,"takePerThrottle":5}
- POST request PROD
  URL- https://ehcuspd1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/prepare/all
  Content-Type: application/json
  Settings - {"throttleMs":5,"takePerThrottle":5}

Rehydrate Stage

- This has to start after #1 completes.

- This is also a background task and the endpoint will return an immediate `202 Accepted`.

Functionality:

- This method facilitates the bulk rehydration of records by fetching the records from cosmos where **rehydrateRequired** is true.
- Creates queue to store the fetched record Ids for batch(takePerThrottle) processing and initiates multiple concurrent task to process record Ids.
- Implements throttling(throttleMs) between the batches of record processing to manage and optimize Cosmos DB RU.
- Fetch the event source message for each id, iterates through the ordered event source messages and apply to the state and produce the applicable events.
- If the event source message is un processable then update the unprocessable status in the respective container.
- Sets the **rehydrateRequired** to false after successfully applying the event source messages.

End Points:

POST request DEV
URL - https://eheu2dv1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/all
Settings - {"throttleMs":5,"takePerThrottle":50,"whereRehydrateRequiredIsTrue":true}

POST request INTG
URL - https://eheu2in1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/all
Settings - {"throttleMs":5,"takePerThrottle":50,"whereRehydrateRequiredIsTrue":true}

POST request PROD
URL - https://ehcuspd1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/all
Settings - {"throttleMs":5,"takePerThrottle":50,"whereRehydrateRequiredIsTrue":true}

For the Decision service rehydration currently , we set the RU for the containers as below

RequestMessages	pd1	768000
RequestState	pd1	768000

Settings for PROD rehydration as below
takePerThrottle - 2500

throttleMs - 1000 milliseconds

The Prepare stage for Decision service runs for 2 days in Prod and then the Rehydration is started

<u>Possible Parallelization approach to speed up the Process for Bulk rehydration:</u>

1. **Create a new Long running AppService for Prepare and Rehydration stage** :
   - Appservice has to be started and stopped manually during rehydration . The prepare and Rehydration action has to be set via appsettings.
   - This will enable us to scale the instance to a practical level(2-3).Because the increase in instance will also lead to increase in Container RU(2-3 times of current Rehydration RU level) as per the instance increase.
   - This can save theoretically minimum 50% rehydration time with 2 instances in rehydration (excluding the prepare stage).
2. **[Recommended]Utilizing Azure Functions for Rehydration Stage:**
   - As per the last discussion with David , we have a upper limit on Cosmos DB RU as 1 million .
   - During rehydration, we are already setting it as 768000 RU, we are left with the option of increasing only addition of ~20000 RUs.
   - So, we can't ideally increase the instance for parallel processing as we have RU constraint
   - The existing API can still be utilized for Prepare stage .For parallel processing ,we can produce a limited information message(RequestforServiceKey,SentTS) to a message queue like Azure service bus.
   - While the Prepare stage is running, The Azure function Can be utilized for Rehydration stage based on the triggers from Prepared requests in message Queue. This parallel processing can save us 2 days time(Prepare stage time).
   - Note:
     - The new Azure function will still be running with 1 instance
     - The existing event source library has to be modified to have new methods to process event source messages with RequestForserviceId as the input from trigger message
     - This method will have multiple roundtrip to DB based on trigger from message queue